PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
Compiladores
2s/2026 – Prof. Lucas Reis

Projeto MICROC – Etapa 6

Geração de pseudoRV64 e seleção por árvores

2º semestre de 2026

Objetivo

O grupo traduzirá a AST validada na Semântica 2 para pseudoRV64, uma representação intermediária de máquina (IR), executável no interpretador fornecido. As operações se aproximam de RISC-V, com registradores virtuais. A alocação física e a geração de assembly ficam para a etapa seguinte.

Resultado da etapa

AST validada $\longrightarrow$ gerador $\longrightarrow$ `ProgramIR`

validação estrutural $\longrightarrow$ inspeção ou interpretação

Os comandos serão traduzidos *ad hoc*, com tratamento específico de cada caso. As expressões usam padrões de árvores (*tree-pattern matching*) e programação dinâmica (DP), com cobertura de custo mínimo no catálogo publicado neste enunciado.

1 Organização e entrega

1.1 Recupere o projeto anterior

Reúna todos os módulos anteriores diretamente em `previous/`. Copie primeiro os módulos e pacotes auxiliares que estavam em `previous/` da Semântica 2. Depois copie os módulos e pacotes da raiz da Semântica 2 para essa mesma pasta. Em conflitos, prevalece a versão da raiz da Semântica 2, incluindo `semantic.py` e `semantic_errors.py`. Não copie a pasta `previous/` como subdiretório.

Preserve os arquivos `load_previous.py` fornecidos nesta etapa. O carregador prioriza a raiz do starter e a pasta única `previous/`. Use o `runner.py` da raiz. Os imports antigos continuam usando seus nomes originais, sem prefixos de pacote. Todas as passagens compartilham as mesmas classes de AST, símbolos e tipos. Não copie Git, ambientes virtuais ou caches. Os pontos pendentes do frontend devem ser substituídos pelo trabalho do grupo.

1.2 Material inicial e entrega

O starter fornece o modelo da IR, execução, validação, impressão textual, contexto de função, criação de operandos, mapa de variáveis por símbolo e representação declarativa de regras. O grupo completa:

- `codegen.py`: percurso de funções e comandos, reconstrução da cobertura e ações que emitem instruções;
- `selector.py`: reconhecimento dos padrões e cálculo dos custos mínimos por nó e classe de resultado; e
- `catalogue.py`: descrição das regras obrigatórias com os tipos fornecidos em `patterns.py`.

Os métodos pendentes lançam `NotImplementedError`. Não os substitua por retornos vazios. O grupo pode criar auxiliares e reorganizar a implementação, preservando as interfaces públicas e as estruturas fornecidas. A infraestrutura não faz a tradução de expressões ou comandos em nome do grupo.

O projeto continua sendo desenvolvido em trios. Não há relatório obrigatório ou bônus por otimizações adicionais.

Prazo desta etapa

A entrega encerra-se no **domingo, 15 de novembro de 2026, às 23h59**, no horário de Brasília. O commit considerado na correção deve estar no repositório correto do grupo no GitHub até esse horário.

2 Interfaces, símbolos e representação

2.1 Entrada e saída do gerador

O pipeline deve executar o frontend completo antes de chamar o gerador:

```
program = Parser(Lexer(source).scan()).parse()
program = SemanticAnalyzer().analyze(program)
ir = CodeGenerator().generate(program)
```

`generate` recebe um `Program` e devolve um `ProgramIR`. Erros léxicos, sintáticos ou semânticos impedem essa chamada. Use as anotações `symbol`, `scope` e `type` existentes. Não refaça as análises ou altere a AST. Gere o mesmo programa repetidamente, inclusive reutilizando o objeto `CodeGenerator`.

O CFG da etapa anterior serviu à análise semântica. Nesta entrega, o gerador percorre a AST e produz listas de instruções por função. Não é necessário converter esse CFG ou criar uma árvore intermediária para o seletor. As regras da linguagem permanecem na especificação normativa da MICROC.

Estrutura	Conteúdo
`ProgramIR`	`functions`: lista de funções. `strings`: mapa de `StringRef` para conteúdo decodificado.
`FunctionIR`	`function`: `FunctionSymbol` original. `instructions`: lista ordenada de instruções.
`VReg`	`owner`: símbolo da função proprietária. `id`: ID local.
`Label`	`owner`: função proprietária. `id`: ID local do destino.
`StringRef`	`id`: ID de uma string no pool do programa.

Um registrador virtual (*virtual register*, VReg) contém 64 bits em uma ativação. VRegs e labels de funções diferentes não são intercambiáveis. Seus IDs impressos não são resultados normativos.

2.2 Contexto e identidade dos símbolos

Crie um `Context(function_ir, program_ir)` para cada função. `temp()` cria um VReg temporário. `label()` cria um label. `emit(instruction)` acrescenta uma instrução à função atual. `string(text)` registra conteúdo decodificado no pool. `variable(symbol)` fornece o VReg associado àquele objeto de símbolo. Os serviços estão implementados, mas o grupo decide quando usá-los. Uma variável usa um VReg mutável, reescrito nas atribuições. Não se exigem SSA, memória por variável ou outra IR de código de três endereços.

O `x` de um bloco interno e o `x` externo possuem símbolos distintos. Consulte o objeto anotado, sem refazer buscas por nome. `variable` não emite instruções nem inicializa valores. Uma declaração sem inicializador não recebe zero implicitamente.

3 Instruções e execução da pseudoRV64

3.1 Valores, controle e origem

As instruções são objetos Python definidos em `ir.py`. Os campos `dst`, `src`, `lhs`, `rhs` e `value` representam VRegs. `imm`, `shift` e `index` são inteiros. Todas as instruções aceitam `origin=SourceSpan` para preservar a posição correspondente na AST. Operações sintéticas sem origem própria, como labels auxiliares, podem usar `None`.

Os nomes da tabela são classes Python. O dump textual usa mnemônicos em minúsculas: `Addi` aparece como `addi`, `Move` como `mv` e `Jump` como `j`. Labels terminam com dois-pontos. `param`, `return` e `print_int`, entre outras, são operações abstratas desta IR. Não representam a convenção de chamada ou o runtime RV64.

Quando há strings, o dump começa pela seção `.strings`, com entradas como `str0: "texto"`. Ela exibe o mapa `ProgramIR.strings`. Aspas, barras e caracteres de controle são escapados para preservar o conteúdo em uma linha. Assim, uma quebra aparece como `\n` no dump.

Instrução	**Efeito**
`Li(dst, imm)`	Carrega uma constante inteira com sinal de 64 bits.
`Move(dst, src)`	Copia o valor atual.
`Neg(dst, src)`	Nega o inteiro, conservando 64 bits.
`Add/Sub/Mul/Div/Rem`	Campos `(dst, lhs, rhs)`. Operação aritmética.
`Addi(dst, src, imm)`	Soma com imediato entre -2048 e 2047.
`Slli(dst, src, shift)`	Desloca à esquerda de 0 a 63 bits.
`Slt(dst, lhs, rhs)`	Escreve 1 se `lhs < rhs` com sinal, ou 0.
`Xor(dst, lhs, rhs)`	Ou-exclusivo dos padrões de 64 bits.
`Seqz/Snez(dst, src)`	Testa igualdade/desigualdade com zero e escreve 0/1.
`LabelInstr(label)`	Marca um destino local à função.
`Jump(target)`	Transfere o controle incondicionalmente.
`Beq/Bne/Blt/Bge`	Campos `(lhs, rhs, target)`. Desvia se a relação vale.
`Beqz/Bnez(src, target)`	Desvia comparando o valor com zero.
`Param(dst, index)`	Recebe o argumento da ativação atual.
`Call(dst, function, args)`	Chama o símbolo com uma lista ordenada de VRegs.
`Return(value)`	Encerra a ativação atual.
`PrintInt/PrintBool(value)`	Imprime o valor sem separador ou quebra adicional.
`PrintString(string)`	Imprime o conteúdo da `StringRef`.

Um desvio condicional (*branch*) salta se sua condição é verdadeira. Caso contrário, a execução segue na instrução seguinte. Um `Jump` possui somente o destino. `Return` não possui continuação na função. Labels não alteram valores. A consulta de leituras e definições, USE/DEF, já está disponível em `uses(instruction)` e `defs(instruction)`. Não implemente vivacidade nesta etapa.

Os operandos são lidos antes da escrita de `dst`. O destino pode coincidir com uma entrada, inclusive em atualizações de variáveis. Operações com `dst` definem esse VReg e leem suas entradas. `Li` e `Param` não leem VRegs. `Call` lê seus argumentos e define somente o destino, quando presente. Branches, retornos e impressão de valores leem seus operandos sem definir VRegs. Labels, `Jump`, `StringRef` e campos imediatos não são leituras de VRegs.

O validador fornecido, `validate(ir)`, confere a estrutura da IR, incluindo operandos, destinos, referências, imediatos e assinaturas. O executor usa ativações independentes e interrompe uma leitura de VReg ainda sem valor no caminho executado. Isso é falha da IR gerada. Queda pelo fim de uma função sem `Return` também é falha da IR, conforme a regra de retorno desta etapa.

3.2 Semântica aritmética e booleanos

A execução segue a especificação MicroC. Soma, subtração, multiplicação e negação conservam os 64 bits menos significativos. A divisão com sinal trunca em direção a zero. O resto não nulo tem o sinal do dividendo. Por exemplo, `-7 / 3` resulta em -2 e `-7 % 3` resulta em -1. O mínimo com sinal dividido por -1 conserva o mínimo, com resto zero.

Essas operações já estão implementadas no interpretador. O grupo deve selecionar as instruções corretas. Não deve calcular seus resultados no Python durante a geração. Divisão e resto por zero continuam indefinidos. Não é necessário inserir verificações em execução. O interpretador pode interromper esses casos para diagnóstico.

Valores `bool` devem ser materializados como 0 ou 1. Essa representação não permite coerções entre `int` e `bool`. Comparações relacionais usam inteiros com sinal. Igualdade e desigualdade também aceitam booleanos, conforme a tipagem anterior.

4 Seleção por padrões de árvores

4.1 Classes de resultado e padrões

Uma mesma expressão pode ser implementada de formas diferentes. Em `x + 5`, o literal pode ser carregado em um VReg ou incorporado ao imediato de `Addi`. O seletor guarda alternativas por classe de resultado, não apenas uma escolha por nó. As classes não são novos tipos da MicroC:

Classe	Resultado oferecido
`REG`	Valor materializado em um VReg.
`COND`	Emissão de controle com destinos verdadeiro e falso.
`IMM12`	Literal reconhecido no intervalo de imediato de `Addi`.
`POW2`	Literal positivo 2^k, oferecendo o expoente k.
`EFFECT`	Chamada executada com resultado descartado.

`REG`, `COND`, `IMM12`, `POW2` e `EFFECT` são constantes fornecidas por `patterns.py`. Um resultado literal de `IMM12` ou `POW2` não emite instruções. `COND` se aplica somente a expressões `bool`. Uma chamada `void` oferece somente `EFFECT`.

Em `Rule`, `name` identifica a regra, `result` é a classe oferecida e `cost` é seu custo próprio inteiro não negativo. `action` identifica uma ação emissora a implementar pelo grupo.

`Pattern(kind, operator, children, predicate)` restringe o tipo do nó, o operador, os filhos e, opcionalmente, uma propriedade da subárvore. Campos `None` não restringem a correspondência. Os padrões usam `kind=None`, `operator=None`, `children=()` e `predicate=None` como valores padrão. A lista de filhos é uma tupla. `Need(classe)` marca o ponto em que outra cobertura será exigida. `Args(REG)`, como único filho do padrão de uma chamada, exige `REG` em cada argumento, inclusive quando a lista é vazia.

A regra abaixo exemplifica a representação de uma única alternativa. Seu corpo de emissão continua sendo responsabilidade do grupo:

```
Rule(
    name="soma_imediato_direita", result=REG,
    pattern=Pattern(BinaryExpr, BinaryOperator.ADD,
                    (Need(REG), Need(IMM12))),
    cost=1, action="emitir_soma_imediato_direita",
)
```

O reconhecimento deve percorrer o padrão sobre a AST original. `children(node)` fornece filhos na ordem da fonte. Um padrão com predicado e sem filhos reconhece a subárvore inteira quando esse predicado é verdadeiro. Nos demais casos, a quantidade de filhos deve coincidir. Não se exigem regras de cadeia entre classes no mesmo nó.

Os predicados fornecidos não avaliam expressões. `signed_literal` reconhece `IntLiteral` ou uma negação unária imediata de `IntLiteral`. Assim, -2048 pode ser reconhecido sem exigir dobramento de constantes (*constant folding*). `imm12` testa o intervalo de 12 bits. `pow2` aceita potências positivas com $0 \leq k \leq 62$. `neg_imm12` reconhece um literal cujo oposto cabe no imediato de `Addi`. Uma subárvore como `2 + 3` não é um literal para esses predicados.

4.2 Tabela de custos e cobertura mínima

A interface pública do seletor é:

```
table = Selector(rules).analyze(expression)
choice = table.lookup(node, result_class)
```

`analyze` deve calcular os custos em pós-ordem, sem emitir instruções ou criar temporários. Para cada nó e classe, considere todas as regras aplicáveis. Uma regra exige correspondência estrutural e cobertura para cada subproblema marcado por `Need`. Descarte alternativas impossíveis.

Se uma regra r pede classes c_i nas subárvores n_i, seu custo total é:

$$C_r(n) = \text{custo}(r) + \sum_i C(n_i, c_i).$$

Guarde o menor custo de cada classe, junto com a regra escolhida e os subproblemas necessários à reconstrução. Não confunda custo próprio com custo total. Um custo menor em outra classe não substitui a classe pedida pelo pai.

`lookup` devolve `Choice(cost, rule, subproblems)` ou `None` se a classe não estiver coberta. `subproblems` é uma tupla ordenada de pares com nó e classe de resultado, na ordem dos pontos `Need` do padrão. `SelectionTable.put` registra uma escolha para o nó e a classe. A tabela usa identidade dos nós e mantém seus próprios dados, sem escrever custos na AST. Empates admitem qualquer cobertura mínima, com escolha estável em execuções repetidas. A emissão de uma raiz sem cobertura deve falhar com mensagem clara que identifique a posição do nó.

4.3 Exemplo: duas coberturas para uma soma

Considere a expressão `x + 5`. A Figura 1 e a tabela abaixo usam o VReg já associado ao símbolo de `x`.

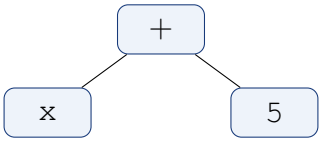


Figura 1: A AST da soma. O literal pode oferecer mais de uma classe de resultado.

Nó	Custo REG	Custo IMM12	Custo POW2
`x`	0	–	–
`5`	1	0	–
`x + 5`	1	–	–

O traço indica ausência de cobertura. `Li` materializa o literal com custo 1. Seu reconhecimento como imediato custa 0. A alternativa geral soma $1 + 0 + 1 = 2$: uma `Add`, o valor de `x` e a carga do literal. A alternativa com imediato soma $1 + 0 + 0 = 1$. A cobertura mínima usa `Addi`.

A reconstrução segue a regra escolhida na raiz e pede as classes exigidas. Não emita a `Li` da alternativa rejeitada. A otimalidade vale somente para a árvore, o catálogo publicado e seus custos aditivos.

5 Reconstrução, condições e ordem de avaliação

5.1 Valores e desvios são resultados diferentes

Uma comparação usada em `bool b = a < c;` precisa produzir 0/1 em VReg. A mesma comparação em `if (a < c)` pode desviar diretamente usando seus operandos. O seletor deve manter essas alternativas como `REG` e `COND`. Uma classe não representa apenas um custo diferente: ela define como o código que a utiliza receberá o resultado.

Na reconstrução de `REG`, uma ação oferece um VReg ao seu usuário. Na reconstrução de `COND`, recebe os destinos verdadeiro e falso e emite o controle correspondente. A assinatura do auxiliar interno é livre. A emissão deve seguir a cobertura selecionada, sem escolher instruções por novos testes que ignorem o DP.

5.2 Curto-circuito e efeitos de chamadas

O cálculo em pós-ordem não executa expressões. Pode examinar os dois filhos de `a && f()` sem chamar `f`. A emissão, porém, não pode gerar antecipadamente todos os filhos na mesma sequência. A ação de `&&` deve colocar a emissão do segundo filho no caminho em que o primeiro foi verdadeiro. Em `||`, esse caminho depende de o primeiro ser falso. `!` pode inverter os destinos de uma condição.

No programa abaixo, a primeira chamada de `marca` é pulada. A segunda é executada porque `a < b` é falso:

```
bool marca() {
    print("M");
    return true;
}

int main() {
    int a = 3;
    int b = 2;
    if (a < b && marca()) {
        print("ramo verdadeiro");
    }
    bool x = a < b || marca();
    print(x);
    return 0;
}
```

A saída é `M`, uma quebra de linha, `true` e outra quebra de linha. O retorno de `main` é zero. Trocar a emissão dos operandos mudaria a saída, mesmo que a operação lógica parecesse equivalente.

Quando `&&` ou `||` é usado como valor, a ação deve organizar caminhos que escrevem 1 ou 0 em um destino e convergem. Não basta devolver um valor diferente de zero. O mesmo cuidado vale em inicializadores, atribuições, argumentos, retornos e itens de impressão.

5.3 Comandos continuam com tradução ad hoc

O percurso dos comandos decide como usar as expressões selecionadas. Um `IfStmt` organiza os destinos dos ramos e sua continuação. Um `WhileStmt` organiza teste, corpo, saída e salto de volta. Essas decisões são implementadas diretamente no gerador, sem regras de cobertura para comandos. A condição é solicitada ao seletor como `COND`.

Blocos da AST são percorridos na ordem dos comandos. Declarações com inicializador e atribuições avaliam o valor antes de escrever no VReg do símbolo. Um `return` encerra o caminho executado. Não é necessário eliminar as instruções inalcançáveis, mas elas não podem ser executadas por uma conexão incorreta de controle. Laços infinitos válidos não precisam produzir retorno.

6 Funções, chamadas e impressão

6.1 Parâmetros, ativações e retorno

Cada função deve ter uma `FunctionIR`. Emita um `Param` para cada parâmetro no início de sua lista, uma vez por índice, seguindo a assinatura. Os índices começam em zero. O símbolo do parâmetro identifica seu VReg, que pode receber atribuições no corpo.

`Call` referencia o `FunctionSymbol` já resolvido. Seus argumentos são VRegs cujos valores foram obtidos na ordem esquerda para a direita. A lista pode conter mais de oito argumentos. Não use registradores físicos, endereços de pilha ou uma convenção de chamada concreta nesta entrega. Uma chamada que produz valor usa `REG`. Uma chamada usada como comando pode solicitar `EFFECT` e emitir `dst=None`.

O interpretador cria uma ativação com valores locais independentes a cada chamada, inclusive recursiva. Os demais VRegs do chamador são preservados.

Retornos `int` e `bool` fornecem um VReg. Retornos `void` usam `Return(None)`. O fim normal de uma função `void` também deve ser representado por essa instrução. Queda pelo fim da lista sem retorno é falha da IR, mesmo em uma função `void`.

6.2 Exemplo: função com soma imediata

```
int incrementar(int x) {
    return x + 5;
}
```

Uma tradução possível do corpo de `incrementar`, no formato textual da IR, é:

```
param v0, 0
addi v1, v0, 5
return v1
```

`Param` e `Return` são emitidos pelo tratamento ad hoc da função e do comando. A `Addi` resulta da cobertura selecionada para a soma. Números de VRegs e temporários auxiliares podem ser diferentes. O arquivo completo `examples/selecao.mc` imprime `12` seguido de uma quebra e retorna zero.

6.3 Itens de impressão e strings

Processe cada item de `print` por completo antes de avançar para o seguinte. Em `print("A", f(), "B")`, o texto `A` deve ser impresso antes de qualquer impressão feita por `f`. Não avalie primeiro todas as expressões para imprimir seus resultados somente depois.

Strings ficam no mapa `ProgramIR.strings`, referenciadas por `StringRef`, sem ocupar VRegs. Já chegam decodificadas pelo parser, incluindo escapes e concatenação de literais adjacentes. Registre esse conteúdo com `Context.string` e emita `PrintString`.

Para expressões, escolha `PrintInt` ou `PrintBool` pelo tipo anotado. Depois do último item, emita uma `PrintString` referenciando a string `"\n"` do pool. Use

`Context.string` para reutilizar essa mesma entrada entre comandos e funções. Assim, cada comando acrescenta exatamente uma quebra final, além de eventuais quebras presentes nos itens.

7 Catálogo obrigatório e custos

Cada instrução executável custa 1. `LabelInstr` custa 0. Conte o código estático da expressão, incluindo ambos os caminhos de uma condição. Não conte ciclos de processador, expansão física ou frequência de execução. Os custos dos comandos ficam fora da conta.

Codifique as famílias abaixo em `catalogue.py`. Os custos são próprios. Some os custos exigidos nas subárvores para obter o total.

Regra e emissão	**Custo**
Literal inteiro com sinal ou bool para REG, com `Li`.	1
Identificador para REG, reutilizando o VReg do símbolo.	0
Literal elegível para IMM12 ou POW2, sem instrução.	0
Negação aritmética de REG para REG, com `Neg`.	1
Aritmética geral de dois REG para REG.	1
Soma REG/IMM12 ou IMM12/REG, com `Addi`.	1
Subtração REG menos literal cujo oposto cabe no imediato, com `Addi`.	1
Multiplicação REG/POW2 ou POW2/REG, com `Slli`.	1
Menor ou maior de dois REG para REG, com `Slt`.	1
Menor ou igual, maior ou igual para REG, com `Slt` e `Seqz`.	2
Igualdade/desigualdade para REG, com `Xor` e `Seqz`/`Snez`.	2
Comparação de dois REG para COND, branch direto e `Jump`.	2
Negação lógica de REG para REG, com `Seqz`.	1
Negação lógica de COND para COND, invertendo destinos.	0
`&&`/`||` de dois COND para COND.	0
`&&`/`||` de dois COND para REG, com duas `Li` e um `Jump`.	3
Literal bool para COND, com `Jump`.	1
Identificador bool para COND, com branch e `Jump`.	2
Chamada com argumentos REG para REG ou EFFECT, com `Call`.	1
Chamada bool com argumentos REG para COND, com chamada e desvios.	3

Cada condição possui destinos verdadeiro e falso explícitos. A comparação em `COND` emite um branch e um salto para o outro destino. A emissão de `&&` e `||` como condição acrescenta labels, mas nenhuma instrução executável própria além das coberturas dos filhos. Como `REG`, acrescenta duas cargas, de 1 e 0, e um salto de junção. Por isso seu custo próprio é 3.

A soma com `Addi` e a multiplicação com `Slli` reconhecem o literal em qualquer lado. `Slli` com deslocamento zero é a regra para multiplicação por um. Não estenda essa regra à divisão com sinal. Os literais com sinal coexistem com a regra geral de negação. Não há folding geral no modo avaliado. Se ambos os operandos forem literais, aplique o mesmo catálogo e aceite empates. Extensões opcionais ficam em modo separado, sem bônus.

Na subtração `x - c`, use `addi` com imediato $-c$ quando $-2048 \leq -c \leq 2047$, isto é, quando $-2047 \leq c \leq 2048$. O predicado `neg_imm12` está fornecido. A regra reconhece diretamente o literal à direita, sem exigir que ele ofereça `IMM12` e sem acrescentar outra classe de resultado. Seu custo próprio é 1, somado ao custo de `x` como `REG`. A alternativa geral acrescenta a carga do literal e uma `Sub`, totalizando 2 além de `x`. O DP deve escolher o mínimo. Por exemplo, `x - 2048` admite imediato -2048, mas `x - (-2048)` exigiria 2048 e usa a alternativa geral. A regra não se aplica a `c - x`. Preserve a avaliação única de `x`.

8 Execução, testes e critérios de conclusão

8.1 Ambiente e modos do runner

Depois de recuperar o frontend, execute a partir da raiz do starter:

```
python -m pip install -r requirements-dev.txt
python runner.py examples/selecao.mc --dump-ir
python runner.py examples/selecao.mc --run
python runner.py examples/curto_circuito.mc --run
python -m pytest -q
python run_tests.py --report resultados.json
```

O ambiente de referência é Python 3.12. O runner mostra a IR por padrão. `-dump-ir` e `-run` são alternativas. O modo de execução escreve somente a saída do programa em stdout. O retorno integral de `main` é o campo `main_return` do resultado de `Interpreter.run(ir)`, sem redução ao status de saída do sistema operacional. O resultado também contém `stdout` e `steps`. `-max-steps` limita a execução de testes.

O runner termina com status 0 no sucesso, 1 para erro do frontend, 2 para uso ou leitura de arquivo, 3 para método pendente e 4 para IR inválida ou falha de execução. Atingir o limite de passos é uma interrupção de teste, não um novo erro semântico da MicroC.

8.2 O que os testes conferem

Comece pelos testes das partes fornecidas:

```
python -m pytest -q tests/test_infrastructure.py
```

Os demais testes públicos descrevem a implementação completa. Eles incluem custos por classe, falta de cobertura, empates, limites de imediato, padrões obrigatórios, comparações, efeitos de chamadas, retorno, escopos, laços, strings e limites numéricos. Falhas iniciais nos pontos pendentes são esperadas. Não use `skip` ou `xfail` para ocultá-las no GitHub Actions.

A correção verifica seleção, estrutura da IR e comportamento, combinando as mesmas regras em outros programas. Não exige dumps ou IDs específicos, nem eliminação de código inalcançável. Verifica os operandos e o efeito das instruções, além de sua presença. Resultado correto não dispensa DP e cobertura mínima.

Não altere AST, símbolos, diagnósticos ou instruções fornecidas. Nesta entrega não implemente alocação física, vivacidade, spills, frames, convenção binária de chamadas, runtime ou emissão de `.S`. A geração termina em `ProgramIR` validado e interpretável.

8.3 Antes da entrega

Confira se:

- todos os módulos do frontend estão diretamente em `previous/`;
- o gerador usa símbolos por identidade e preserva a AST e suas anotações;
- todas as funções possuem parâmetros e retornos compatíveis;
- o seletor considera alternativas e guarda mínimos por nó e classe;
- a reconstrução emite somente a cobertura escolhida;
- condições usam destinos corretos e booleanos materializados são 0/1;
- curto-circuito e ordem de avaliação valem em todos os contextos;
- chamadas, impressão, laços e símbolos sombreados mantêm seu comportamento;
- os custos do catálogo correspondem à emissão das expressões;
- os testes públicos passam sem métodos pendentes; e
- o último commit está no repositório correto e foi conferido no GitHub Actions.

9 Referências para consulta

Consulte a especificação normativa da MICROC já distribuída à turma. Este enunciado reúne o contrato da IR e o catálogo desta etapa. A Aula 10 apresenta procedimentos e registros de ativação. As Aulas 11 e 12 discutem forma do código, expressões e controle. As Aulas 13 e 14 apresentam seleção de instruções e cobertura de árvores com DP. Os acessos à memória usados como exemplos de seleção nas aulas não acrescentam operações de memória para variáveis ao projeto desta etapa.